

ГУАП

КАФЕДРА № 41

ОТЧЕТ
ЗАЩИЩЕН С ОЦЕНКОЙ
ПРЕПОДАВАТЕЛЬ

канд. техн. наук, доцент

должность, уч. степень, звание

подпись, дата

Е. Л. Турнецкая

инициалы, фамилия

ОТЧЕТ О ЛАБОРАТОРНОЙ РАБОТЕ № 1

СОЗДАНИЕ ПРОЕКТА NEXT.JS С ИСПОЛЬЗОВАНИЕМ REACT,
TYPESCRIPT И APP ROUTER

по курсу:

WEB-ПРОГРАММИРОВАНИЕ

РАБОТУ ВЫПОЛНИЛ

СТУДЕНТ гр. № 4326

подпись, дата

Г. С. Томчук

инициалы, фамилия

Санкт-Петербург 2026

1 Цель работы

Цель работы: подготовить среду разработки и создать начальную версию веб-приложения на основе Next.js, React и TypeScript с использованием App Router.

2 Описание концепции приложения

В ходе выполнения лабораторной работы была разработана начальная версия учебного приложения QuestCity. Приложение задумано как агрегатор городских квестов, объединяющий сведения о предложениях разных организаторов в одном интерфейсе. На первом этапе реализован информационный каркас проекта: главная страница, страница «О сервисе», навигация и представление трёх будущих возможностей сервиса.

3 Решаемая проблема, целевая аудитория и предметная сущность

Информация о городских квестах часто распределена между сайтами отдельных организаторов. Из-за этого пользователю приходится самостоятельно искать площадки, открывать несколько ресурсов и сравнивать жанры, стоимость, сложность, продолжительность, число игроков и другие характеристики. QuestCity должен сократить количество таких действий: собрать предложения в одном веб-приложении и предоставить понятный интерфейс для поиска и выбора подходящего варианта.

Целевая аудитория приложения — жители города и туристы, заинтересованные в посещении квестов. Сервис ориентирован на компании друзей, пары, семьи и отдельных пользователей, которые хотят выбрать развлечение с учётом интересующих характеристик.

Главной предметной сущностью приложения является квест. В последующих версиях с этой сущностью могут быть связаны название, описание, изображения, жанр, адрес, организатор, допустимое количество игроков, продолжительность, уровни сложности и страха, возрастное ограничение, цена, рейтинг, отзывы и доступные сеансы.

Первоначальная концепция предусматривает следующие пользовательские возможности:

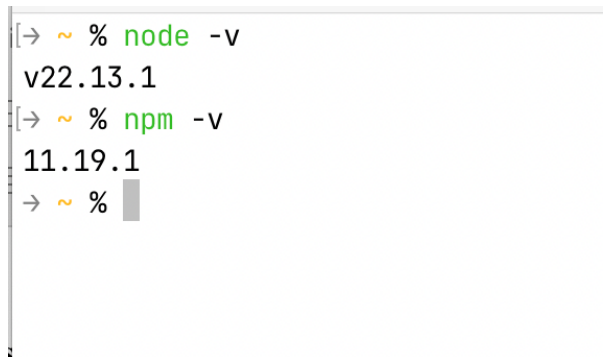
1. просмотр каталога городских квестов;
2. поиск и подбор квестов по характеристикам;
3. сохранение интересных квестов в избранное.

Дальнейшее развитие проекта предполагает создание каталога, динамических страниц отдельных квестов, поиска, фильтрации, отзывов, бронирования, учётных записей и личного кабинета.

4 Ход выполнения

4.1 Создание нового приложения Next.js

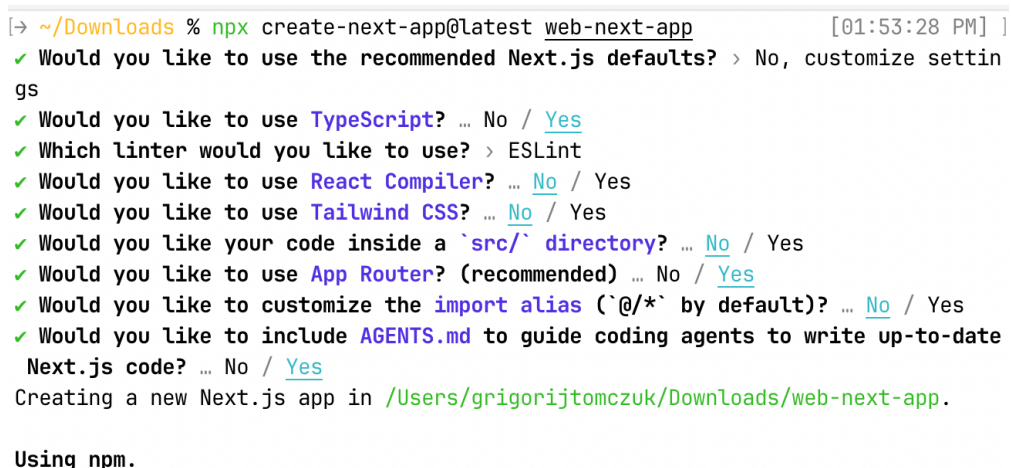
Для выполнения работы использовались среда выполнения Node.js версии v22.13.1 и пакетный менеджер npm версии 11.19.1. На рисунке 1 представлено отображение версий Node.js и npm, использованных при выполнении работы.



```
[→ ~ % node -v
v22.13.1
[→ ~ % npm -v
11.19.1
→ ~ %
```

Рисунок 1 — Версии Node.js и npm

При создании проекта были выбраны следующие параметры (рисунок 2):



```
[→ ~/Downloads % npx create-next-app@latest web-next-app [01:53:28 PM] ]
✓ Would you like to use the recommended Next.js defaults? > No, customize settings
✓ Would you like to use TypeScript? ... No / Yes
✓ Which linter would you like to use? > ESLint
✓ Would you like to use React Compiler? ... No / Yes
✓ Would you like to use Tailwind CSS? ... No / Yes
✓ Would you like your code inside a `src/` directory? ... No / Yes
✓ Would you like to use App Router? (recommended) ... No / Yes
✓ Would you like to customize the import alias (`@/*` by default)? ... No / Yes
✓ Would you like to include AGENTS.md to guide coding agents to write up-to-date Next.js code? ... No / Yes
Creating a new Next.js app in /Users/grigorijtomczuk/Downloads/web-next-app.

Using npm.
```

Рисунок 2 — Параметры инициализатора create-next-app

4.2 Определение файловой структуры проекта

Каталог проекта имеет следующую структуру (рисунок 3):

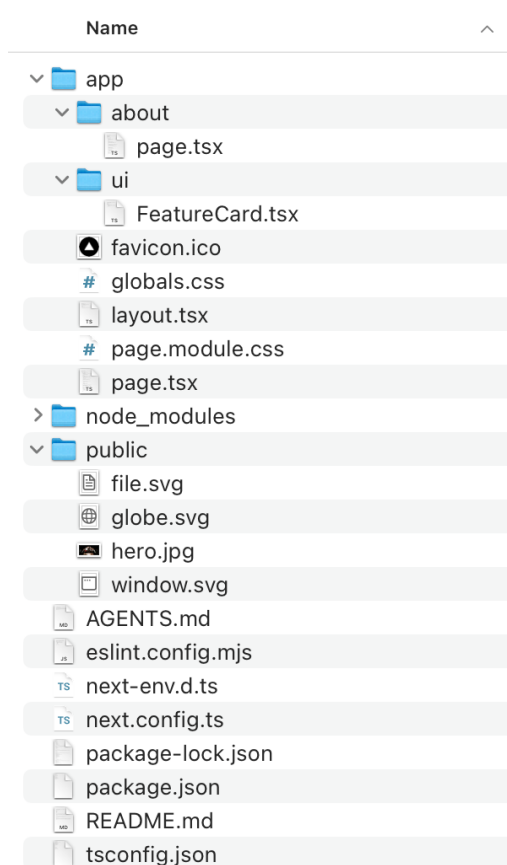


Рисунок 3 — Файловая структура проекта

Файл `app/layout.tsx` содержит корневой макет, обязательные элементы `html` и `body`, атрибут `lang="ru"`, импорт глобальной таблицы стилей и метаданные приложения. В метаданных указаны название `QuestCity` и описание «Агрегатор городских квестов». Файл `app/page.tsx` формирует корневой маршрут `«/»`, а файл `app/about/page.tsx` — маршрут `«/about»`. Переиспользуемые элементы интерфейса размещены в каталоге `app/ui`. Оформление обеих страниц определено в `app/globals.css`.

4.3 Создание первого React-компонента

Для отображения предполагаемых возможностей сервиса был создан компонент `FeatureCard`. Его свойства описаны типом `FeatureCardProps` и обернуты в `Readonly`. Компонент принимает обязательные строковые свойства `title` и `description`, благодаря чему TypeScript проверяет корректность его использования. Семантический элемент `article` позволяет

воспринимать каждую карточку как самостоятельный информационный блок. Листинг 1 содержит код компонента FeatureCard.

Листинг 1 — Компонент FeatureCard

```
// Readonly не позволяет компоненту изменять полученные свойства.
type FeatureCardProps = Readonly<{
  title: string;
  description: string;
}>;

export default function FeatureCard({
  title,
  description,
}: FeatureCardProps) {
  return (
    // Article сохраняет смысл карточки как самостоятельного материала.
    <article className="feature-card">
      <span className="feature-card__marker" aria-hidden="true" />
      <h3>{title}</h3>
      <p>{description}</p>
    </article>
  );
}
```

4.4 Реализация главной страницы

Главная страница приложения QuestCity реализована в файле `app/page.tsx` и доступна по маршруту `/`. Она представляет собой основной информационный экран агрегатора городских квестов.

В верхней части страницы расположен заголовочный блок с названием QuestCity и навигационной ссылкой на страницу «О сервисе». Для перехода между маршрутами используется компонент `Link` из библиотеки `next/link`, обеспечивающий навигацию без полной перезагрузки страницы.

Главный экран содержит название сервиса, краткое описание его назначения и кнопку перехода к дополнительной информации. Справа размещено тематическое изображение участников квеста. Изображение хранится в каталоге `public`, подключается с помощью компонента `Image` из `Next.js` и загружается приоритетно, поскольку находится в первом видимом блоке страницы. Для него также заданы альтернативный текст и адаптивные размеры.

Ниже расположен раздел «Возможности». В нём представлены

просмотр каталога квестов, поиск подходящего варианта и сохранение понравившихся квестов. Для отображения этих возможностей трижды используется переиспользуемый компонент FeatureCard, которому передаются различные значения типизированных свойств title и description.

В нижней части страницы находится информационный блок, объясняющий пользу сервиса, и подвал со ссылкой на страницу «О сервисе». Для разметки применены семантические элементы main, section, header, nav, figure, article и footer.

Главная страница является серверным компонентом. Использование директивы "use client" не требуется, поскольку страница не содержит сложного состояния, обработчиков событий или другой клиентской логики.

На рисунке 4 представлена главная страница приложения QuestCity.

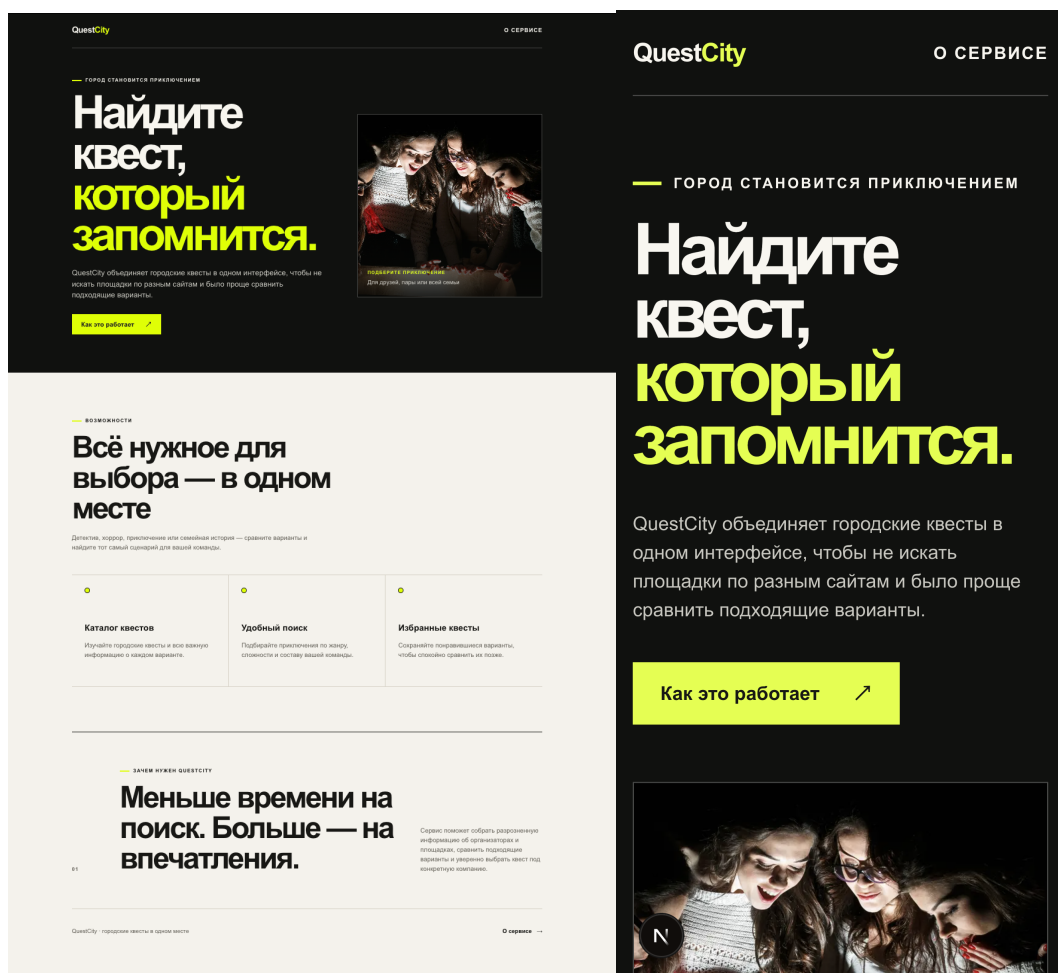


Рисунок 4 — Главная страница

4.5 Реализация страницы «О сервисе»

Страница «О сервисе» реализована в файле `app/about/page.tsx` и доступна по маршруту `/about`. Она содержит дополнительную информацию о назначении и преимуществах сервиса QuestCity.

В заголовочной части страницы размещены название приложения и навигационная ссылка для возврата на главную страницу. Переход также реализован с помощью компонента `Link` из `next/link`.

Основной блок объясняет идею QuestCity как единого сервиса, собирающего информацию о городских квестах разных организаторов. Содержимое страницы ориентировано на пользователя и рассказывает о преимуществах поиска квестов через единый интерфейс.

Далее представлены три тематических блока. Первый описывает возможность сравнивать предложения разных организаторов в одном месте. Второй посвящён выбору подходящего приключения для пары, семьи или компании друзей. Третий содержит характеристики, на которые рекомендуется обращать внимание при выборе квеста: жанр и атмосфера, состав команды, уровень сложности, продолжительность, расположение и возрастные ограничения.

Перечень характеристик хранится в массиве `selectionTips`. Элементы списка создаются с помощью метода `map`, что позволяет избежать повторения одинаковой JSX-разметки. Текст каждого пункта используется в качестве уникального ключа `key`.

Страница построена с применением семантических элементов и остаётся серверным компонентом. В нижней части размещена ссылка для возврата на главную страницу приложения.

На рисунке 5 представлена страница «О сервисе».

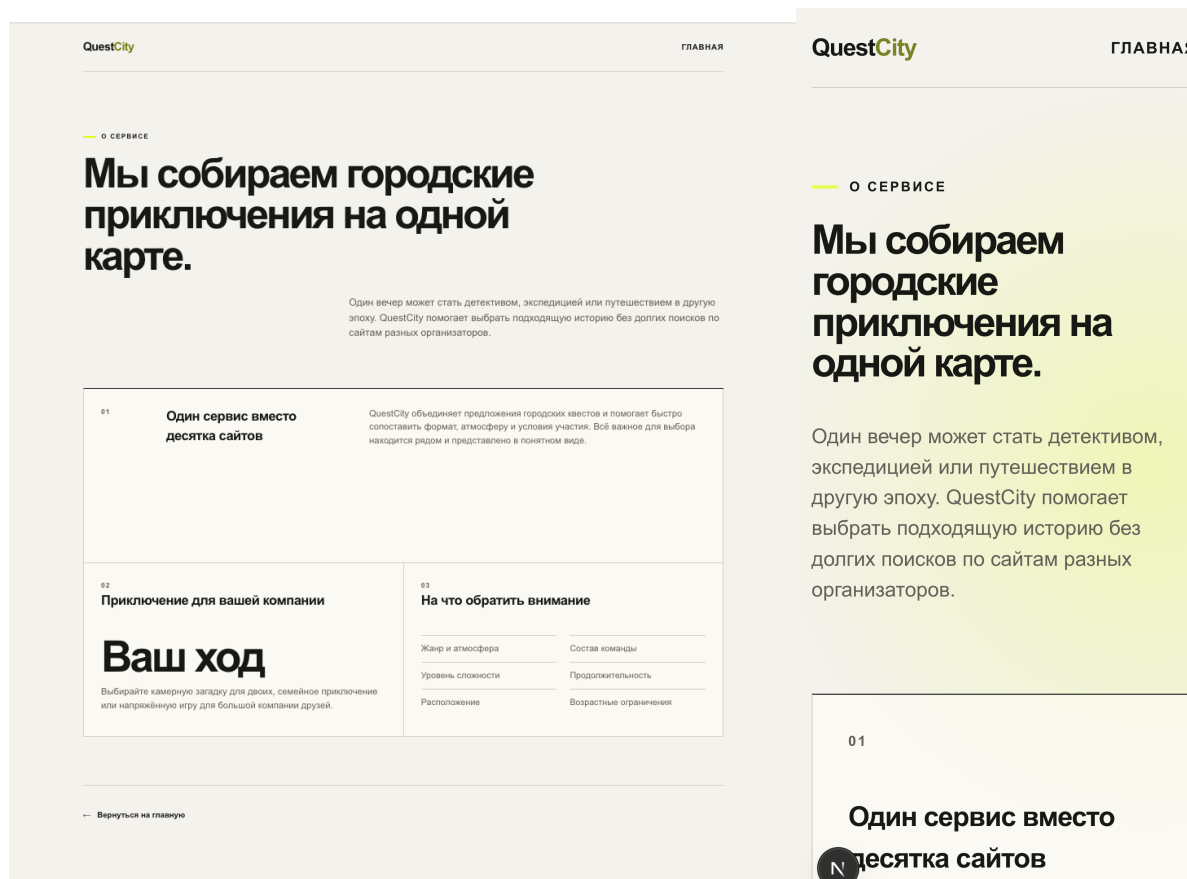


Рисунок 5 — Страница «О сервисе»

4.6 Проверка и сборка

После завершения реализации в каталоге `app` была выполнена статическая проверка командой: `npm run lint`. Команда завершилась успешно, ошибок ESLint обнаружено не было. Затем была запущена производственная сборка: `npm run build`.

Сборка завершилась успешно. Next.js скомпилировал проект, выполнил проверку TypeScript и статически сформировал маршруты `«/»` и `«/about»`. В итоговой таблице маршрутов обе страницы отмечены как статически предварительно отрисованные. На рисунке 5 представлен результат выполнения команды `npm run build`.

```
→ ~/Desktop/suai/web/app (git:main) % npm run build

> questcity@0.1.0 build
> next build

▲ Next.js 16.3.4 (Turbopack)
✓ Running next.config.ts took 15ms

Creating an optimized production build ...
✓ Compiled successfully in 240ms
✓ Finished TypeScript in 816ms
✓ Collecting page data using 6 workers in 350ms
✓ Generating static pages using 6 workers (5/5) in 325ms
✓ Finalizing page optimization in 6ms

Route (app)
├ ○ /
├ ○ /_not-found
└ ○ /about

○ (Static) prerendered as static content
```

Рисунок 6 — Сборка приложения

5 Выводы

В ходе лабораторной работы была подготовлена начальная версия веб-приложения QuestCity на основе Next.js, React и TypeScript. Маршруты главной и информационной страниц организованы с помощью App Router, переходы между ними реализованы компонентом Link. Был создан и трижды использован переиспользуемый компонент FeatureCard с типизированными свойствами. Интерфейс оформлен обычным адаптивным CSS без сторонних UI-библиотек и клиентского состояния. Успешное выполнение ESLint и производственной сборки подтверждает корректность начального каркаса проекта, который будет последовательно расширяться в следующих лабораторных работах.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Next.js. Installation: официальная документация / Vercel. — URL: <https://nextjs.org/docs/app/getting-started/installation> (дата обращения: 10.09.2026).
2. React. Describing the UI: официальная документация / Meta Platforms, Inc. — URL: <https://react.dev/learn/describing-the-ui> (дата обращения: 10.09.2026).
3. TypeScript Handbook: официальная документация / Microsoft. — URL: <https://www.typescriptlang.org/docs/handbook/intro.html> (дата обращения: 10.09.2026).

ПРИЛОЖЕНИЕ

Исходный код

Листинг 2 — Главная страница приложения QuestCity

```
import Image from "next/image";
import Link from "next/link";

import FeatureCard from "@app/ui/FeatureCard";

export default function Home() {
  return (
    <main>
      <section className="hero">
        <header className="site-header container">
          <Link className="brand" href="/" aria-label="QuestCity – на главную">
            Quest<span>City</span>
          </Link>
          <nav aria-label="Основная навигация">
            <Link className="nav-link" href="/about">
              О сервисе
            </Link>
          </nav>
        </header>

        {/* Hero объединяет основное предложение сервиса и тематический визуал. */}
        <div className="hero__content container">
          <div className="hero__copy">
            <p className="eyebrow">Город становится приключением</p>
            <h1>
              Найдите квест,
              <span> который запомнится.</span>
            </h1>
            <p className="hero__description">
              QuestCity объединяет городские квесты в одном интерфейсе, чтобы
              не искать площадки по разным сайтам и было проще сравнить
              подходящие варианты.
            </p>
            <Link className="primary-link" href="/about">
              Как это работает <span aria-hidden="true">👉</span>
            </Link>
          </div>

          <figure className="hero__visual">
            {/* Priority ускоряет загрузку изображения в первом экране. */}
            <Image
              src="/hero.jpg"
              alt="Команда участников разгадывает тайну квеста"
              fill
              priority
              sizes="(max-width: 820px) 100vw, 42vw"
            />
            <figcaption>
              <span>Подберите приключение</span>
              Для друзей, пары или всей семьи
            </figcaption>
          </figure>
        </div>
      </section>
```

```

<section className="features container" aria-labelledby="features-title">
  <div className="section-heading">
    <p className="eyebrow">Возможности</p>
    <h2 id="features-title">Всё нужное для выбора – в одном месте</h2>
    <p>
      Детектив, хоррор, приключение или семейная история – сравните
      варианты и найдите тот самый сценарий для вашей команды.
    </p>
  </div>

  {/* Один компонент получает разные данные через типизированные props. */}
  <div className="feature-grid">
    <FeatureCard
      title="Каталог квестов"
      description="Изучайте городские квесты и всю важную информацию о каждом
варианте."
    </>
    <FeatureCard
      title="Удобный поиск"
      description="Подбирайте приключения по жанру, сложности и составу вашей
команды."
    </>
    <FeatureCard
      title="Избранные квесты"
      description="Сохраняйте понравившиеся варианты, чтобы спокойно сравнить
их позже."
    </>
  </div>
</section>

<section className="mission container" aria-labelledby="mission-title">
  <p className="mission__index">01</p>
  <div>
    <p className="eyebrow">Зачем нужен QuestCity</p>
    <h2 id="mission-title">
      Меньше времени на поиск. Больше – на впечатления.
    </h2>
  </div>
  <p>
    Сервис поможет собрать разрозненную информацию об организаторах и
    площадках, сравнить подходящие варианты и уверенно выбрать квест под
    конкретную компанию.
  </p>
</section>

<footer className="site-footer container">
  <p>QuestCity · городские квесты в одном месте</p>
  <Link className="text-link" href="/about">
    О сервисе <span aria-hidden="true">→</span>
  </Link>
</footer>
</main>
);
}

```

Листинг 3 — Страница «О сервисе»

```

import Link from "next/link";

// Массив позволяет выводить однотипные пункты без повторения разметки.
const selectionTips = [
  "Жанр и атмосфера",

```

```

    "Состав команды",
    "Уровень сложности",
    "Продолжительность",
    "Расположение",
    "Возрастные ограничения",
  ];

export default function AboutPage() {
  return (
    <main className="about-page">
      <header className="site-header container">
        <Link className="brand" href="/" aria-label="QuestCity – на главную">
          Quest<span>City</span>
        </Link>
        <nav aria-label="Основная навигация">
          <Link className="nav-link" href="/">
            Главная
          </Link>
        </nav>
      </header>

      <section className="about-hero container">
        <p className="eyebrow">0 сервисе</p>
        <h1>Мы собираем городские приключения на одной карте.</h1>
        <p className="about-lead">
          Один вечер может стать детективом, экспедицией или путешествием в
          другую эпоху. QuestCity помогает выбрать подходящую историю без
          долгих поисков по сайтам разных организаторов.
        </p>
      </section>

      {/* Карточки группируют преимущества и ориентиры для выбора квеста. */}
      <section className="about-grid container" aria-label="Преимущества сервиса">
        <article className="about-card about-card--wide">
          <p className="section-number">01</p>
          <h2>Один сервис вместо десятка сайтов</h2>
          <p>
            QuestCity объединяет предложения городских квестов и помогает
            быстро сопоставить формат, атмосферу и условия участия. Всё важное
            для выбора находится рядом и представлено в понятном виде.
          </p>
        </article>

        <article className="about-card">
          <p className="section-number">02</p>
          <h2>Приключение для вашей компании</h2>
          <p className="entity-name">Ваш ход</p>
          <p>
            Выбирайте камерную загадку для двоих, семейное приключение или
            напряжённую игру для большой компании друзей.
          </p>
        </article>

        <article className="about-card">
          <p className="section-number">03</p>
          <h2>На что обратить внимание</h2>
          <ul className="section-list">
            {/* Текст пункта уникален, поэтому используется как стабильный key. */}
            {selectionTips.map((tip) => (
              <li key={tip}>{tip}</li>
            ))}
          </ul>
        </article>
      </section>
    </main>
  );
}

```

```
        </article>
    </section>

    <footer className="about-footer container">
        <Link className="text-link" href="/">
            <span aria-hidden="true"></span> Вернуться на главную
        </Link>
    </footer>
</main>
);
}
```